AWS SFTP Malware Router — Step-by-Step Deployment Guide

This guide walks through every step required to deploy the Healthworks SFTP Malware Router pipeline on AWS. Follow each step in order.

Architecture Overview

```
SFTP Client ──► AWS Transfer Family ──► S3 Inbound Bucket (dev-sftp-inbound-quarantine)
                                                  │
                                       GuardDuty Malware Scan
                                                  │
                                           EventBridge Rule
                                                  │
                                           Lambda Function
                                             ┌─────┴─────┐
                                        CLEAN             INFECTED
                                          │                    │
                              Production Bucket     Quarantine Bucket
                                                  +  SNS Alert Email
```

Flow Summary:

1. An SFTP user uploads a `.pdf` file to the inbound S3 bucket via AWS Transfer Family.
2. Amazon GuardDuty automatically scans the uploaded object for malware.
3. An EventBridge rule captures the GuardDuty scan result and triggers a Lambda function.
4. The Lambda function routes the file:
   - **Clean files** → Production bucket (`healthworks-lambda-manage-productionencounterbucke-yoqwb36evr2x`)
   - **Infected files** → Quarantine bucket (`dev-sftp-infected-quarantine`) + SNS email alert

---

Prerequisites

- AWS Account with Administrator or sufficient IAM privileges
- AWS CLI configured (`aws configure`) — Region: `us-east-1`
- Python 3.12 runtime available for Lambda
- Client's public SSH key (for SFTP user setup)
- Email addresses for the Healthcare IT/Security team (for SNS alerts)

Step 1: Create the S3 Buckets

You need **three** S3 buckets. Create them in the `us-east-1` region.

1.1 — Inbound Quarantine Bucket (SFTP Landing Zone)

1. Go to **Amazon S3 Console** → **Create bucket**.
2. **Bucket name:** `dev-sftp-inbound-quarantine`
3. **Region:** US East (N. Virginia) `us-east-1`

4. **Encryption:** Enable **SSE-S3** (Amazon S3 Managed Keys).
5. **Block Public Access:** Keep all options **enabled** (block all public access).
6. Click **Create bucket**.

> *This is where SFTP users upload files. GuardDuty will scan objects here.*

1.2 — Production Bucket (Clean Files Destination)

1. Go to **Amazon S3 Console** → **Create bucket**.
2. **Bucket name:** `healthworks-lambda-manage-productionencounterbucke-yoqwb36evr2x`
3. **Region:** US East (N. Virginia) `us-east-1`
4. **Encryption:** Enable **SSE-S3**.
5. **Block Public Access:** Keep all options **enabled**.
6. Click **Create bucket**.
7. Go to the bucket → **Permissions** tab → **Bucket Policy** → click **Edit**.
8. Paste the following policy (from `S3-bucket/sftp_production_bucket_policy.json`):

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "DenyInsecureTransport",
            "Effect": "Deny",
            "Principal": "*",
            "Action": "s3:*",
            "Resource": [
                "arn:aws:s3:::healthworks-lambda-manage-productionencounterbucke-
yoqwb36evr2x",
                "arn:aws:s3:::healthworks-lambda-manage-productionencounterbucke-
yoqwb36evr2x/*"
            ],
            "Condition": {
                "Bool": {
                    "aws:SecureTransport": "false"
                }
            }
        },
        {
            "Sid": "AllowMalwareRouterWrite",
            "Effect": "Allow",
            "Principal": {
                "AWS": "arn:aws:iam::744640651507:role/dev-SFTP-Malware-Router-Role"
            },
            "Action": [
                "s3:PutObject",
                "s3:PutObjectTagging",
                "s3:ListBucket",
                "s3:GetBucketLocation"
            ],
            "Resource": [
                "arn:aws:s3:::healthworks-lambda-manage-productionencounterbucke-
yoqwb36evr2x",
```

```
                "arn:aws:s3:::healthworks-lambda-manage-productionencounterbucke-
yoqwb36evr2x/*"
            ]
        }
    ]
}
```

9. Click **Save changes**.

> *This policy enforces SSL/TLS-only access and grants write permission only to the Lambda execution role.*

1.3 — Infected Quarantine Bucket

1. Go to **Amazon S3 Console** → **Create bucket**.
2. **Bucket name:** `dev-sftp-infected-quarantine`
3. **Region:** US East (N. Virginia) `us-east-1`
4. **Encryption:** Enable **SSE-S3**.
5. **Block Public Access:** Keep all options **enabled**.
6. Click **Create bucket**.
7. Go to the bucket → **Permissions** tab → **Bucket Policy** → click **Edit**.
8. Paste the following policy (from `S3-bucket/sftp_infected_quarantine_bucket policy.json`):

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "RestrictAccessToAuthorizedRolesOnly",
            "Effect": "Deny",
            "Principal": "*",
            "Action": "s3:*",
            "Resource": [
                "arn:aws:s3:::dev-sftp-infected-quarantine",
                "arn:aws:s3:::dev-sftp-infected-quarantine/*"
            ],
            "Condition": {
                "ArnNotLike": {
                    "aws:PrincipalArn": [
                        "arn:aws:iam::744640651507:role/dev-SFTP-Malware-Router-Role",
                        "arn:aws:iam::744640651507:user/kasuns@champsoft.com",
                        "arn:aws:iam::744640651507:user/umesh@champsoft.com",
                        "arn:aws:iam::744640651507:user/naduni@champsoft.com"
                    ]
                }
            }
        }
    ]
}
```

9. Click **Save changes**.

> *This policy denies all access except for the Lambda role and three authorized IAM users. Update the authorized users list as needed.*

Step 2: Enable GuardDuty Malware Protection for S3

1. Go to **Amazon GuardDuty Console**.
2. If GuardDuty is not already enabled, click **Enable GuardDuty**.
3. In the left menu, go to **Malware Protection** → **Malware Protection for S3**.
4. Click **Enable** and configure it to scan the `dev-sftp-inbound-quarantine` bucket.
5. GuardDuty will now automatically scan every new object uploaded to this bucket.

Step 3: Create IAM Roles and Policies

3.1 — Lambda Execution Role (`dev-SFTP-Malware-Router-Role`)

This role is assumed by the Lambda function for S3 operations, SNS publishing, and CloudWatch logging.

3.1.1 — Create the Role

1. Go to **IAM Console** → **Roles** → **Create role**.
2. **Trusted entity type:** AWS Service.
3. **Use case:** Lambda.
4. Click **Next**.

3.1.2 — Attach Policy: Basic Execution (CloudWatch Logs)

1. Click **Create policy** (opens in a new tab).
2. Switch to the **JSON** tab and paste (from `iam-policies/lambda_basic_execution_role.json`):

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "logs:CreateLogGroup",
                "logs:CreateLogStream",
                "logs:PutLogEvents"
            ],
            "Resource": "*"
        }
    ]
}
```

3. Click **Next** → Name: `dev-SFTP-Lambda-Basic-Execution` → click **Create policy**.

3.1.3 — Attach Policy: File Transfer (S3 + SNS)

1. Click **Create policy** again.
2. Switch to the **JSON** tab and paste (from `iam-policies/file_transfer_policy.json`):

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
```

```
            "Sid": "AllowBucketListing",
            "Effect": "Allow",
            "Action": [
                "s3:ListBucket",
                "s3:GetBucketLocation"
            ],
            "Resource": [
                "arn:aws:s3:::dev-sftp-inbound-quarantine",
                "arn:aws:s3:::healthworks-lambda-manage-productionencounterbucke-
yoqwb36evr2x",
                "arn:aws:s3:::dev-sftp-infected-quarantine"
            ]
        },
        {
            "Sid": "AllowFileOperations",
            "Effect": "Allow",
            "Action": [
                "s3:PutObject",
                "s3:GetObject",
                "s3:GetObjectVersion",
                "s3:DeleteObject",
                "s3:PutObjectTagging"
            ],
            "Resource": [
                "arn:aws:s3:::dev-sftp-inbound-quarantine/*",
                "arn:aws:s3:::healthworks-lambda-manage-productionencounterbucke-
yoqwb36evr2x/*",
                "arn:aws:s3:::dev-sftp-infected-quarantine/*"
            ]
        },
        {
            "Sid": "SNSAlerts",
            "Effect": "Allow",
            "Action": "sns:Publish",
            "Resource": "arn:aws:sns:us-east-1:744640651507:dev-SFTP-Malware-Detected-Alert"
        }
    ]
}
```

3. Click **Next** → Name: `dev-SFTP-File-Transfer-Policy` → click **Create policy**.

3.1.4 — Complete Role Creation

1. Go back to the **Create role** tab.
2. Search for and select both policies:
   - `dev-SFTP-Lambda-Basic-Execution`
   - `dev-SFTP-File-Transfer-Policy`
3. Click **Next**.
4. **Role name:** `dev-SFTP-Malware-Router-Role`
5. Click **Create role**.

3.2 — SFTP User Access Role

This role is attached to the SFTP user in AWS Transfer Family. It restricts them to PDF-only uploads.

1. Go to **IAM Console** → **Roles** → **Create role**.
2. **Trusted entity type:** AWS Service.
3. **Use case:** Transfer (AWS Transfer Family).
4. Click **Next**.
5. Click **Create policy** and paste (from `iam-policies/SFTP_S3_Inbound_Access_Policy.json`):

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "AllowOnlyPDFUploads",
            "Effect": "Allow",
            "Action": [
                "s3:PutObject",
                "s3:PutObjectTagging"
            ],
            "Resource": [
                "arn:aws:s3:::dev-sftp-inbound-quarantine/healthworks/*/*.pdf",
                "arn:aws:s3:::dev-sftp-inbound-quarantine/healthworks/*/*.PDF"
            ]
        },
        {
            "Sid": "AllowUserFolderListing",
            "Effect": "Allow",
            "Action": [
                "s3:ListBucket",
                "s3:GetBucketLocation"
            ],
            "Resource": "arn:aws:s3:::dev-sftp-inbound-quarantine"
        },
        {
            "Sid": "AllowReadAccess",
            "Effect": "Allow",
            "Action": [
                "s3:GetObject",
                "s3:GetObjectVersion"
            ],
            "Resource": "arn:aws:s3:::dev-sftp-inbound-quarantine/healthworks/*"
        }
    ]
}
```

6. Click **Next** → Name: `dev-SFTP-S3-Inbound-Access-Policy` → click **Create policy**.
7. Go back to the **Create role** tab and attach `dev-SFTP-S3-Inbound-Access-Policy`.
8. **Role name:** `dev-SFTP-Transfer-Service-Role`
9. Click **Create role**.

> *This policy enforces that SFTP users can **only upload** `.pdf` / `.PDF` **files** under the `healthworks/` prefix. Any other file type upload will receive an **Access Denied** error.*

3.3 — EventBridge Invoke Lambda Role

This role allows EventBridge to invoke the Lambda function.

3.3.1 — Create the Trust Policy (Role)

1. Go to **IAM Console** → **Roles** → **Create role**.
2. **Trusted entity type:** Custom trust policy.
3. Paste the trust policy (from `iam-policies/Amazon_EventBridge_Invoke_Lambda_trust_policy.json`):

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "TrustEventBridgeService",
            "Effect": "Allow",
            "Principal": {
                "Service": "events.amazonaws.com"
            },
            "Action": "sts:AssumeRole",
            "Condition": {
                "StringEquals": {
                    "aws:SourceAccount": "744640651507",
                    "aws:SourceArn": "arn:aws:events:us-east-1:744640651507:rule/Dev-
Trigger-SFTP-Malware-Router"
                }
            }
        }
    ]
}
```

4. Click **Next**.

3.3.2 — Create and Attach the Permission Policy

1. Click **Create policy** and paste (from `iam-policies/Amazon_EventBridge_Invoke_Lambda_role.json`):

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "lambda:InvokeFunction"
            ],
            "Resource": [
                "arn:aws:lambda:us-east-1:744640651507:function:dev-SFTP-Malware-Router"
            ]
        }
```

```
    ]
}
```

2. Click **Next** → Name: `dev-EventBridge-Invoke-Lambda-Policy` → click **Create policy**.
3. Go back and attach this policy.
4. **Role name:** `dev-EventBridge-Invoke-SFTP-Lambda-Role`
5. Click **Create role**.

Step 4: Create the SNS Topic

1. Go to **Amazon SNS Console** → **Topics** → **Create topic**.
2. **Type:** Standard.
3. **Name:** `dev-SFTP-Malware-Detected-Alert`
4. Click **Create topic**.
5. Note the Topic ARN: `arn:aws:sns:us-east-1:744640651507:dev-SFTP-Malware-Detected-Alert`
6. Go to **Subscriptions** → **Create subscription**.
7. **Protocol:** Email.
8. **Endpoint:** Enter the Healthcare IT/Security team email address.
9. Click **Create subscription**.
10. Repeat for each team member email.
11. Each subscriber **must confirm** by clicking the link in the confirmation email they receive.

> *When malware is detected, all confirmed subscribers will receive an alert email with the infected filename and quarantine details.*

Step 5: Deploy the Lambda Function

5.1 — Create the Function

1. Go to **AWS Lambda Console** → **Create function**.
2. **Function name:** `dev-SFTP-Malware-Router`
3. **Runtime:** Python 3.12
4. **Architecture:** x86_64
5. **Execution role:** Use existing role → select `dev-SFTP-Malware-Router-Role`
6. Click **Create function**.

5.2 — Upload the Code

1. In the Lambda function editor, replace the default code with the contents of `lambda/lambda_function.py`:

```python
import boto3
import urllib.parse
import logging
import os

logger = logging.getLogger()
logger.setLevel(logging.INFO)

s3 = boto3.client('s3')
sns = boto3.client('sns')
```

```python
CLEAN_BUCKET = 'healthworks-lambda-manage-productionencounterbucke-yoqwb36evr2x'
INFECTED_BUCKET = 'dev-sftp-infected-quarantine'
SNS_TOPIC_ARN = 'arn:aws:sns:us-east-1:744640651507:dev-SFTP-Malware-Detected-Alert'

def get_unique_key(bucket, key):
    """
    Checks if a file exists in the target bucket.
    If it does, appends a numeric suffix (e.g., _1, _2) until a unique key is found.
    """
    base_name, extension = os.path.splitext(key)
    final_key = key
    counter = 1

    while True:
        try:
            s3.head_object(Bucket=bucket, Key=final_key)
            final_key = f"{base_name}_{counter}{extension}"
            counter += 1
        except:
            break
    return final_key

def lambda_handler(event, context):
    logger.info(f"EVENT: {event}")
    try:
        detail = event.get('detail', {})

        s3_info = detail.get('s3ObjectDetails') or detail.get('resourceDetails',
{}).get('s3BucketDetails')

        if not s3_info:
            logger.error("No S3 details found in event. Check EventBridge Pattern or Test
JSON.")
            return {"status": "ERROR", "message": "Key mismatch in JSON"}

        source_bucket = s3_info['bucketName']
        original_key = urllib.parse.unquote_plus(s3_info['objectKey'])
        scan_result = detail.get('scanResultDetails', {}).get('scanResultStatus')

        if scan_result == "NO_THREATS_FOUND":
            logger.info(f"Processing clean file: {original_key}")
            final_key = get_unique_key(CLEAN_BUCKET, original_key)
            s3.copy_object(
                Bucket=CLEAN_BUCKET,
                CopySource={'Bucket': source_bucket, 'Key': original_key},
                Key=final_key,
                TaggingDirective='REPLACE'
            )
            s3.delete_object(Bucket=source_bucket, Key=original_key)
            logger.info(f"SUCCESS: Moved {original_key} to Production as {final_key}")
            return {"status": "SUCCESS", "destination": final_key}
```

```
        elif scan_result == "THREATS_FOUND":
            logger.warning(f"MALWARE DETECTED in: {original_key}")
            final_key = get_unique_key(INFECTED_BUCKET, original_key)
            s3.copy_object(
                Bucket=INFECTED_BUCKET,
                CopySource={'Bucket': source_bucket, 'Key': original_key},
                Key=final_key,
                TaggingDirective='REPLACE'
            )
            s3.delete_object(Bucket=source_bucket, Key=original_key)
            sns.publish(
                TopicArn=SNS_TOPIC_ARN,
                Subject="MALWARE ALERT",
                Message=f"Infected file detected and quarantined.\nOriginal Name:
{original_key}\nStored as: {final_key}"
            )
            logger.info(f"QUARANTINED: {original_key} moved as {final_key}")
            return {"status": "QUARANTINED", "destination": final_key}

    except Exception as e:
        logger.error(f"CRITICAL ERROR: {str(e)}")
        raise e
```

5.3 — Update Environment Variables (if needed)

Before deploying, verify these three variables at the top of the code match your environment:

Variable	Value
`CLEAN_BUCKET`	`healthworks-lambda-manage-productionencounterbucke-yoqwb36evr2x`
`INFECTED_BUCKET`	`dev-sftp-infected-quarantine`
`SNS_TOPIC_ARN`	`arn:aws:sns:us-east-1:744640651507:dev-SFTP-Malware-Detected-Alert`

5.4 — Configure Timeout

1. Go to **Configuration** → **General configuration** → **Edit**.
2. Set **Timeout** to at least **30 seconds** (recommended for S3 copy operations on larger files).
3. Click **Save**.

5.5 — Deploy

1. Click **Deploy** in the Lambda code editor.

Step 6: Create the EventBridge Rule

6.1 — Create the Rule

1. Go to **Amazon EventBridge Console** → **Rules** → **Create rule**.
2. **Name:** `Dev-Trigger-SFTP-Malware-Router`
3. **Event bus:** default

4. **Rule type:** Rule with an event pattern
5. Click **Next**.

6.2 — Define the Event Pattern

1. Select **Custom pattern (JSON editor)**.
2. Paste the following (from `eventbridge/event_pattern.json`):

```
{
  "source": ["aws.guardduty"],
  "detail-type": ["GuardDuty Malware Protection Object Scan Result"],
  "detail": {
    "s3ObjectDetails": {
      "bucketName": ["dev-sftp-inbound-quarantine"]
    }
  }
}
```

3. Click **Next**.

> *This pattern captures only GuardDuty malware scan results for objects in the `dev-sftp-inbound-quarantine` bucket.*

6.3 — Set the Target

1. **Target type:** AWS service.
2. **Select a target:** Lambda function.
3. **Function:** `dev-SFTP-Malware-Router`
4. **Execution role:** Use existing role → select `dev-EventBridge-Invoke-SFTP-Lambda-Role`
5. Click **Next** → **Next** → **Create rule**.

Step 7: Set Up AWS Transfer Family (SFTP Server)

7.1 — Create the SFTP Server

1. Go to **AWS Transfer Family Console** → **Create server**.
2. **Protocol:** SFTP
3. **Identity provider:** Service managed
4. **Endpoint type:** Public (or VPC if required by your network policy)
5. **Domain:** Amazon S3
6. **Logging role:** Create or select a role that can write to CloudWatch Logs.
7. Click **Create server**.
8. Note the **Server ID** and **Endpoint** (e.g., `s-xxxxxxxxxxxx.server.transfer.us-east-1.amazonaws.com`).

7.2 — Add an SFTP User

1. Go to the SFTP server → **Add user**.
2. **Username:** (e.g., `healthworks-user`)
3. **Role:** Select `dev-SFTP-Transfer-Service-Role` (created in Step 3.2)
4. **Home directory:**
   - **Bucket:** `dev-sftp-inbound-quarantine`
   - **Prefix (optional):** `healthworks/<username>/uploads`

- **Restricted:** Yes (check the box to restrict user to their home directory)

5. **SSH public keys:** Paste the client's **public SSH key** (e.g., contents of `id_rsa.pub`).
6. Click **Add**.

7.3 — Share Connection Details with the Client

Provide the client with:

Detail	Value
Protocol	SFTP
Host	`s-xxxxxxxxxxxx.server.transfer.us-east-1.amazonaws.com`
Port	22
Username	`healthworks-user`
Authentication	SSH private key (matching the imported public key)
Allowed files	`.pdf` / `.PDF` only

Appendix A: Generating SSH Key Pairs for SFTP Authentication

AWS Transfer Family uses **SSH key-based authentication only** (no passwords). The client must generate an SSH key pair and share the **public key** with the administrator for import into the SFTP server.

Option 1: Windows (Using PuTTYgen)

1. Download **PuTTYgen** from https://www.puttygen.com (or install the full PuTTY suite).
2. Open **PuTTYgen**.
3. At the bottom, set **Type of key to generate** to **RSA** and **Number of bits** to **4096**.
4. Click **Generate** and move your mouse randomly over the blank area until the progress bar completes.
5. Once generated:
   - **(Optional)** Enter a **Key passphrase** for extra security.
   - The public key is displayed in the top text box.
6. Click **Save private key** → save as `healthworks_sftp_key.ppk` (this is the client's private key for WinSCP/PuTTY).
7. To get the **OpenSSH-format public key** (required by AWS Transfer Family):
   - In PuTTYgen, go to **Conversions** → **Export OpenSSH key** if you need the private key in OpenSSH format.
   - Copy the **entire** text from the top text box (starts with `ssh-rsa ...`). This is the public key.
8. Save the public key text to a file named `healthworks_sftp_key.pub`.

Option 2: Windows 10/11 (Using Built-in OpenSSH)

1. Open **PowerShell** or **Command Prompt**.
2. Run:

```
ssh-keygen -t rsa -b 4096 -f C:\Users\%USERNAME%\.ssh\healthworks_sftp_key
```

3. When prompted for a passphrase, either enter one or press **Enter** for none.
4. Two files are created:

- `healthworks_sftp_key` — **Private key** (keep this secret, never share)
- `healthworks_sftp_key.pub` — **Public key** (share this with the administrator)

5. View the public key:

```
Get-Content C:\Users\%USERNAME%\.ssh\healthworks_sftp_key.pub
```

Option 3: macOS / Linux (Using Terminal)

1. Open **Terminal**.
2. Run:

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/healthworks_sftp_key
```

3. When prompted for a passphrase, either enter one or press **Enter** for none.
4. Two files are created:
   - `~/.ssh/healthworks_sftp_key` — **Private key** (keep this secret)
   - `~/.ssh/healthworks_sftp_key.pub` — **Public key** (share with administrator)
5. View the public key:

```
cat ~/.ssh/healthworks_sftp_key.pub
```

What the Client Should Share

The client must send **only the public key** to the AWS administrator. The public key looks like:

```
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAACAQ... (long string) ...== user@hostname
```

> ***IMPORTANT:*** *The client must **never share the private key**. The private key stays on the client's machine and is used to authenticate when connecting via SFTP.*

How the Administrator Imports the Public Key

1. Go to **AWS Transfer Family Console** → select the SFTP server → select the user.
2. Under **SSH public keys**, click **Add SSH public key**.
3. Paste the entire public key string (starting with `ssh-rsa`).
4. Click **Add key**.

Connecting with the Private Key

Using WinSCP (Windows)

1. Open **WinSCP** → **New Session**.
2. **File protocol:** SFTP
3. **Host name:** `s-xxxxxxxxxxxx.server.transfer.us-east-1.amazonaws.com`
4. **Port:** 22
5. **User name:** `healthworks-user`
6. Click **Advanced** → **SSH** → **Authentication**.
7. **Private key file:** Browse and select `healthworks_sftp_key.ppk`
   - If you generated an OpenSSH key (Option 2/3), WinSCP will offer to convert it to `.ppk` format automatically.

8. Click **OK** → **Login**.

Using FileZilla (Windows / macOS / Linux)

1. Open **FileZilla** → **Edit** → **Settings** → **SFTP** → **Add key file**.
2. Select the private key file (`healthworks_sftp_key` or `.ppk`).
3. Click **OK**.
4. In the **Quick Connect** bar:
   - **Host:** `sftp://s-xxxxxxxxxxxx.server.transfer.us-east-1.amazonaws.com`
   - **Username:** `healthworks-user`
   - **Port:** 22
5. Click **Quickconnect**.

Using Command Line (macOS / Linux / Windows PowerShell)

```
sftp -i ~/.ssh/healthworks_sftp_key healthworks-user@s-xxxxxxxxxxxx.server.transfer.us-east-1.amazonaws.com
```

Once connected, upload a PDF:

```
put /path/to/medical_report.pdf
```

Step 8: End-to-End Testing & Verification

Test 1: File Extension Restriction

1. Using an SFTP client (e.g., WinSCP, FileZilla, `sftp` CLI), connect to the SFTP server.
2. Attempt to upload a non-PDF file (e.g., `image.png`).
3. **Expected result:** Upload fails with **Access Denied / Permission Denied** error.

Test 2: Successful PDF Upload (Clean File)

1. Upload a legitimate, clean PDF file (e.g., `medical_report.pdf`).
2. Wait 1–3 minutes for GuardDuty to scan the file.
3. **Verify:**
   - The file is **removed** from `dev-sftp-inbound-quarantine` .
   - The file **appears** in `healthworks-lambda-manage-productionencounterbucke-yoqwb36evr2x` with the same key.
   - Check **CloudWatch Logs** for the Lambda function → log entry showing `SUCCESS` .

Test 3: Duplicate File Handling (Collision Avoidance)

1. Upload `medical_report.pdf` again (same filename as Test 2).
2. Wait for the scan and routing to complete.
3. **Verify:**
   - The second file is saved as `medical_report_1.pdf` in the production bucket.
   - Upload a third time → saved as `medical_report_2.pdf` .

Test 4: Malware Detection

1. Create a file containing the **EICAR test string** (standard anti-malware test):

```
X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
```

2. Save it as `test_malware.pdf`.
3. Upload via SFTP.
4. Wait 1–3 minutes for GuardDuty scan.
5. **Verify:**
   - The file is **removed** from `dev-sftp-inbound-quarantine`.
   - The file **appears** in `dev-sftp-infected-quarantine`.
   - An **SNS email alert** is received with subject **"MALWARE ALERT"** containing the filename.
   - Check **CloudWatch Logs** → log entry showing `QUARANTINED`.

Step 9: Monitoring & Ongoing Operations

CloudWatch Logs

- **Log group:** `/aws/lambda/dev-SFTP-Malware-Router`
- Monitor for `SUCCESS`, `QUARANTINED`, and `CRITICAL ERROR` entries.

CloudWatch Alarms (Recommended-not implemented)

1. Go to **CloudWatch** → **Alarms** → **Create alarm**.
2. Create a metric filter on the Lambda log group for the pattern `CRITICAL ERROR`.
3. Set a notification to the SNS topic or a separate Ops topic.

S3 Lifecycle Policies (Recommended-not implemented)

- **Quarantine bucket:** Consider adding a lifecycle rule to auto-delete quarantined files after 90 days.
- **Production bucket:** Consider transitioning old files to S3 Glacier after a retention period.

Resource Summary

Resource	Name / ARN
SFTP Server	`Dev-SFTP-Server`
Inbound S3 Bucket	`dev-sftp-inbound-quarantine`
Production S3 Bucket	`healthworks-lambda-manage-productionencounterbucke-yoqwb36evr2x`
Quarantine S3 Bucket	`dev-sftp-infected-quarantine`
Lambda Function	`dev-SFTP-Malware-Router`
Lambda Role	`dev-SFTP-Malware-Router-Role`
SFTP User Role	`dev-SFTP-Transfer-Service-Role`
EventBridge Rule	`Dev-Trigger-SFTP-Malware-Router`
EventBridge Role	`dev-EventBridge-Invoke-SFTP-Lambda-Role`
SNS Topic	`dev-SFTP-Malware-Detected-Alert`
AWS Account	`744640651507`

Region	`us-east-1`

Troubleshooting

Issue	Check
SFTP upload denied for `.pdf`	Verify the SFTP user role has the correct policy attached. Check the S3 prefix path matches `healthworks/*/`.
File stays in inbound bucket	Verify GuardDuty Malware Protection for S3 is enabled on the bucket. Check the EventBridge rule is active.
Lambda not triggered	Check EventBridge rule status (must be **Enabled**). Verify the event pattern matches the GuardDuty event. Check the EventBridge invocation role.
Lambda errors in CloudWatch	Check Lambda execution role has all required S3 and SNS permissions. Verify bucket names and SNS ARN in the code.
No SNS email received	Confirm email subscriptions are in **Confirmed** status in the SNS console. Check spam/junk folder.
Duplicate filenames not incrementing	Verify Lambda has `s3:HeadObject` (via `s3:GetObject`) permission on the target bucket.